AGENTS.md — Spot-Welder multi-firmware workspace

Guidance for AI coding agents working in this repo. Read this before editing.

Repository map

This is a **mono-repo** containing multiple independent firmware/software projects that communicate over a shared serial/TCP telemetry protocol.

Path	Target	Toolchain	Role
`ESP32P4/`	ESP32-P4 (CrowPanel Advance 5”)	**ESP-IDF** (CMake)	UI + WiFi bridge + STM32/OTA flasher. Talks to STM32 over UART3-IN @ 576000. Enriches STATUS, relays WAVEFORM_* raw.
`STM32G474CE/`	STM32G474CE	**PlatformIO** (`env:g474ceu6_stlink`)	Real-time weld controller: FET drive, ADC/shunt, INA226, charger, joule integrator. **Sole producer of STATUS, WELD_DONE, WAVEFORM_* packets.**
`ESP32_8048S043C/`	ESP32-8048S043C (old board)	**PlatformIO / Arduino**	Legacy display firmware. Produces DISPLAY and CELLS packets. Keep buildable; not the primary target.
(separate repo) `Spot-Welder-Server/`	Python 3 / Flask	`pip`	Dashboard + TCP client to the P4 bridge (:8888). Re-parses telemetry.

> The Flask server lives in a **separate repository** (`Spot-Welder-Server`). For any change that touches the wire protocol, that repo MUST be edited in the same coordinated change.

⚠️ Source of truth: the telemetry protocol

There is currently **no shared protocol header**. The `STATUS` / `WAVEFORM_DATA` packet is:

- **Emitted** in `STM32G474CE/src/main.c` (search `"STATUS,"`) as an ASCII `key=value` CSV line.
- **Parsed** in `ESP32P4/main/wifi_bridge.cpp` and `welder_main.cpp` (`sscanf` / `strtok`).
- **Parsed again** in `Spot-Welder-Server/app.py`.

See **`PROTOCOL.md`** at the repo root for a descriptive reference of observed packet formats.

Rule: any change to a field name, order, type, or units is a breaking change across all three.
When editing the protocol, update ALL of: the STM32 emitter, both P4 parsers, and the Flask parser — and keep this list in sync. Prefer additive changes (append new fields) over reordering.

Build & validation commands

Always build the project(s) you touched before claiming a change is done.

ESP32-P4 (ESP-IDF)

```
cd ESP32P4
idf.py set-target esp32p4      # first time only
idf.py build                   # compile — MUST pass with no errors
idf.py -p <PORT> flash monitor # flash + serial (hardware only)
```

- Output binary: `build/esp32_firmware.bin` (project name is `esp32_firmware`).
- Do NOT flash `hello_world.bin` — that name is stale/removed.

STM32G474CE (PlatformIO)

```
cd STM32G474CE
pio run                             # compile env:g474ceu6_stlink — MUST pass
pio run -t upload                   # flash via ST-Link (hardware only)
pio device monitor -b 576000        # serial monitor at app baud
```

Legacy ESP32-8048S043C (PlatformIO)

```
cd ESP32_8048S043C
pio run                             # keep it compiling; secondary priority
```

Flask server (separate repo)

```
cd Spot-Welder-Server
pip install -r requirements.txt
python app.py
# serves dashboard on :8080, TCP bridge client to P4 :8888
```

Hardware / safety constraints the agent MUST respect

- **UART link is 576000 8N1** through BSS138 level shifters on the UART3-IN header (GPIO 27/28 on P4,
  PA9/PA10 on STM32). Do NOT raise the baud rate — higher rates corrupt through the RC-limited

shifters.
Bootloader mode uses 115200 8E1.

- **WiFi is an external XIAO ESP32-C6 on SPI (J7 header)**, not the board's onboard C6. The onboard C6 uses
SDIO, which collides with the SD card's SDMMC controller; the external C6-over-SPI workaround frees the SD
card. Pins: MOSI 48 / MISO 47 / CLK 26 / CS 29 / HS 30 / DR 31 / reset 32 (active-low), SPI mode 3.
Do NOT raise `CONFIG_ESP_HOSTED_SPI_FREQ_ESP32C6` above 10 MHz (GPIO-matrix slave sampling limit). The C6
transport lives in `ESP32P4/sdkconfig.defaults`; full writeup in `docs/hardware/ESP32C6_WIFI_DAUGHTERBOARD.md`.
- **The charge MOSFET (`CHARGER_EN`) is owned by the STM32** and MUST be OFF during a weld pulse — the
joule integrator assumes discharge-only current. Never move this control to the P4.
- **Reserved:** do not bind host ports 1000/2200 in any tooling.

Conventions

- C/C++: match existing style in each project (no reformatting unrelated code).
- Guard all float telemetry math with `isfinite()` + a `< 0` floor, as the STM32 code already does.
- Never commit, push, open a PR, merge, or auto-merge unless I explicitly request it. When changes are complete, provide a per-repository diff summary.
- After any protocol edit, state explicitly which of the 3 parsers you updated.